



AI Driven Cloud Data Engineering

Create the Azure ML Pipeline

Student Name : Elena Kroupkin

Student ID : 270255004

Contents

- Overview and Setup 3
- Machine Learning Pipeline: Auto Price Training 4
- Data Transformations 7
- Model Training & Evaluation 12
- Validation and Submission 16
- Check Model’s outcome 20
- Explanation for Each Regression Metric 24

Overview and Setup

Step	What to Do	How to Do It	Settings
1.	Open Azure Portal	https://portal.azure.com/auth/login/	
2.	Create a Resource Group	In the search bar, type and select " Resource groups "	- Resource group name: rg-ml-270255004 - Tags: Location, nz
3.	Create a Machine Learning Workspace	In the search bar, type and select " Azure Machine Learning " resource. Then, select " Create new workspace "	- Resource group: rg-ml-270255004 - Name: ws-ml-270255004 - Region: Australia East - Tags: Location, nz
4.	Open ML Studio	Open the " Studio web URL " link or click the " Launch Studio " button	

Screenshot 58: Workspace

Azure AI | Machine Learning Studio

Search

All workspaces

⌵

⚙️

🔍

?

😊

EK

☰

Default Directory

Workspaces

Feature stores

Hubs

Registries

Shared assets

Components

Environments

Models

Default Directory > Workspaces

Workspaces

A workspace provides a centralized place to keep track of all the artifacts you create while performing machine learning experiments.

+ New

Refresh

Edit workspace

Reset view

Search

Filter

Columns

Name ↑	Resource group	Region	Subscription ↓
ws-ml-270255004	rg-ml-270255004	australiaeast	Elena-Subscr-2

Machine Learning Pipeline: Auto Price Training

This table provides a step-by-step guide to building a Linear Regression model in Azure ML Studio to predict car prices, designed for easy understanding and replication.

Preparation & Setup

Pipeline Initialization & Data Loading

Step	What to Do	How to Do It	Settings
1.	Create New Pipeline	In Azure ML Studio, navigate to the Designer section and select "Create New Pipeline."	Assign a descriptive name (e.g., "car_price_prediction").
2.	Load Raw Automobile Price Data	In the Designer canvas, from the right-hand pane , navigate to the Component tab. Expand " Sample data ", then locate automobile_price_raw . Drag this dataset onto the pipeline canvas.	- Sample data: automobile_price_raw directly from Azure's default data.

Screenshot 59: Automobile_price_raw directly from Azure's default data.

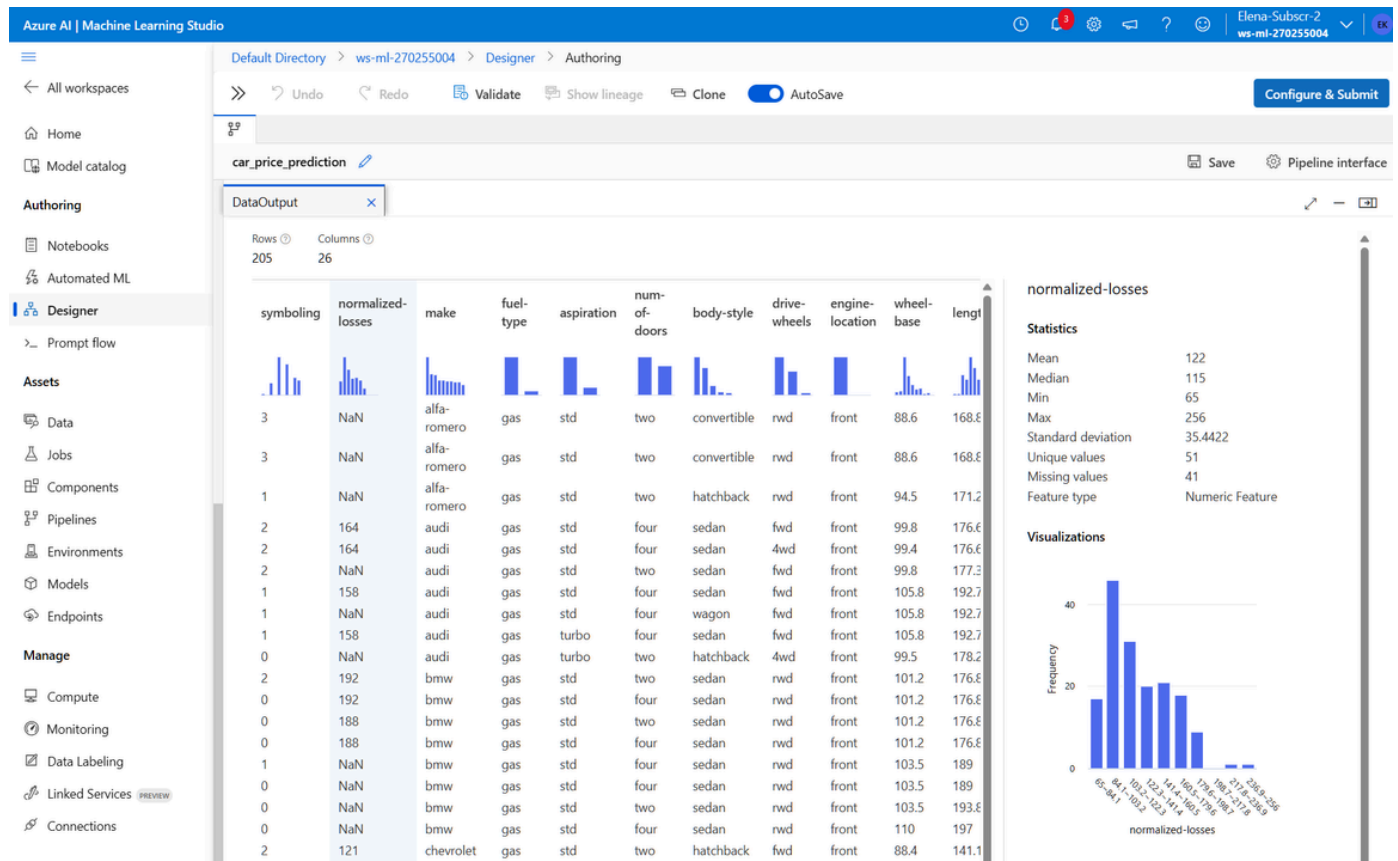
The screenshot displays the Azure ML Studio Designer interface. The left sidebar shows the navigation pane with the 'Designer' section selected. The main canvas area shows the 'Sample data (16)' component list. The 'Automobile price data (Raw)' dataset is highlighted, and a tooltip is visible over it, showing the dataset's description and a 'Use data' button. The dataset name 'car_price_prediction' is visible in the top right corner of the canvas area.

3.	Visualize Data	Right-click the output of the "automobile price data" module	Inspect column distributions, data types,
----	-----------------------	--	---

and select "**Preview data.**"

and check for missing values (e.g., for 'normalized-losses', which often contains NaN values).

Screenshot 60: Preview data - data visualization



4. Configure Azure ML Compute Cluster

(This is configured during pipeline submission, **not a module**). During the "Set up pipeline job" step, select or create a compute cluster.

Provision a dedicated, scalable compute cluster for training operations.

- In Azure ML Studio,
- navigate to **Compute > Compute clusters**.
- Select **+ New**,
- specify a low-cost, multi-core VM size (e.g., Standard_DS11_v2),
- and set the **Minimum number of nodes** to 0 and **Maximum number of nodes** to 4.

Data Cleaning & Pre-processing (Data Transformations)

Column Selection and Missing Data Handling

Step	What to Do	How to Do It	Why It's Important
1.	Select Columns in Dataset	Connect the "Automobile price data" output to a " Select Columns in Dataset " module.	in Select columns click Edit column. • Select: by name • Add all • remove : normalized-loses (as it contains NaN values) • Click Save

Screenshot 61: Select Columns in Dataset

The screenshot displays the Dataiku pipeline interface. At the top, a blue header bar contains navigation icons and user information: 'Elena-Subscr-2' and 'ws-ml-270255004'. Below the header, a blue 'AutoSave' toggle is active, and a 'Configure & Submit' button is visible. The main workspace shows a pipeline diagram with two modules: 'automobile_price_raw' (outputting 'Data output') and 'Select Columns in Dataset' (receiving 'Dataset' input and outputting 'Results dataset'). The 'Select Columns in Dataset' module is selected, opening its configuration panel on the right. The panel has a title 'Select Columns in Datas...' and an 'Edit column' link. Under 'Select columns', a list of column names is shown: 'symboling, make, fuel-type, aspiration, num-of-doors, body-style, drive-wheels, engine-location, wheel-base, length, width, height, curb-weight, engine-type, num-of-'. Below this, 'Output settings' and 'Input settings' sections are visible with expandable arrows.

2.	Clean Missing Data	Connect the output from "Select Columns in Dataset" to a "Clean Missing Data" module.	1. Columns to be cleaned click Edit column: • With rule • All columns • save 2. Cleaning mode: Remove entire row
----	---------------------------	---	--

Screenshot 62: Clean Missing Data

🕒🔔⚙️📢❓😊

Elena-Subscr-2
ws-ml-270255004

▼EK

📄 Clone

☒ AutoSave

Configure & Submit

💾 Save

⚙️ Pipeline interface

📄 automobile_price_raw
automobile_price_raw

V | 3

Data output

Dataset

📄 Select Columns in Dataset
select_columns_in_dataset

Results dataset

Dataset

📄 Clean Missing Data
clean_missing_data

Cleaned datas... Cleaning tran...

Parameters

Clean Missing Data

Columns to be cleaned ⓘ * [Edit column](#)

All columns

Minimum missing value ratio ⓘ * ...

0.0

Maximum missing value ratio ⓘ * ...

1.0

Cleaning mode ⓘ * ...

Remove entire row ▼

Output settings >

Input settings >

Run settings >

Node information >

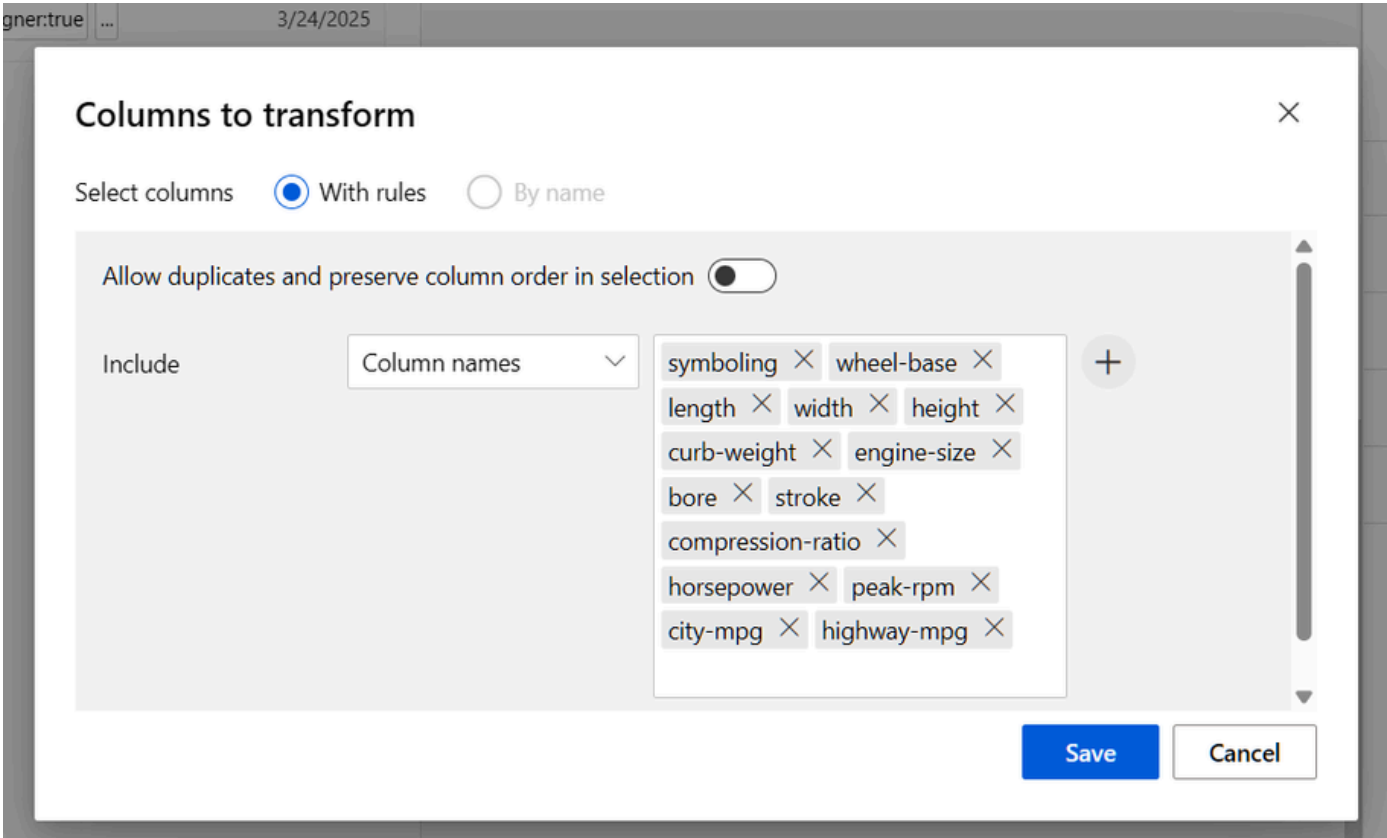
Component information >

Data Normalization

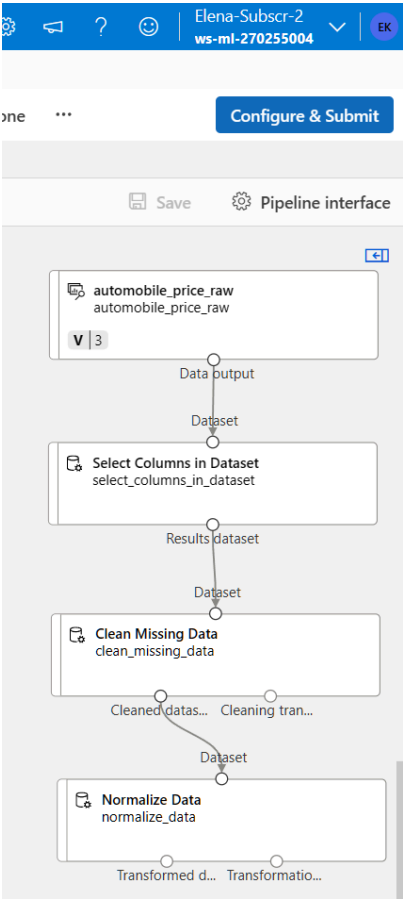
3.	Normalize Data	Connect the cleaned data to a "Normalize Data" module.	1. Transformation method : ZScore 2. Columns to transform • With rules
----	-----------------------	--	---

- type numeric column

Screenshot 63: Normalize Data



Screenshot 64: Pipeline up to Normalize Data



Data Splitting

4.	Split Data	Connect the "Transformed dataset" output (left side) from "Normalize Data" to a "Split Data" module.	Fraction of rows in the first output dataset 0.7 (70%) . This creates two outputs: 70% for training and 30% for testing
----	-------------------	--	---

Screenshot 65: Split Data Settings

The screenshot displays the Data Science Canvas interface. The top navigation bar includes icons for clock, notifications, settings, help, and a user profile (Elena-Subscr-2, ws-ml-270255004). Below the navigation bar, a toolbar contains 'Redo', 'Validate', 'Show lineage', 'Clone', and a 'Configure & Submit' button. The main workspace shows a pipeline with the following steps: 'automobile_price_raw' (dataset), 'Select Columns in Dataset' (transform), 'Clean Missing Data' (transform), 'Normalize Data' (transform), and 'Split Data' (split). The 'Split Data' module is highlighted with a blue border. The right sidebar shows the 'Split Data' configuration panel with the following settings: 'Splitting mode' set to 'Split Rows', 'Fraction of rows in the first output dataset' set to '0.7', 'Randomized split' set to 'True', 'Random seed' set to '0', and 'Stratified split' set to 'False'. Below these settings are expandable sections for 'Output settings', 'Input settings', 'Run settings', 'Node information', and 'Component information'.

Split Data

Splitting mode ⁱ * ...
Split Rows

Fraction of rows in the first output dataset...
0.7

Randomized split ⁱ * ...
True

Random seed ⁱ * ...
0

Stratified split ⁱ * ...
False

Output settings >

Input settings >

Run settings >

Node information >

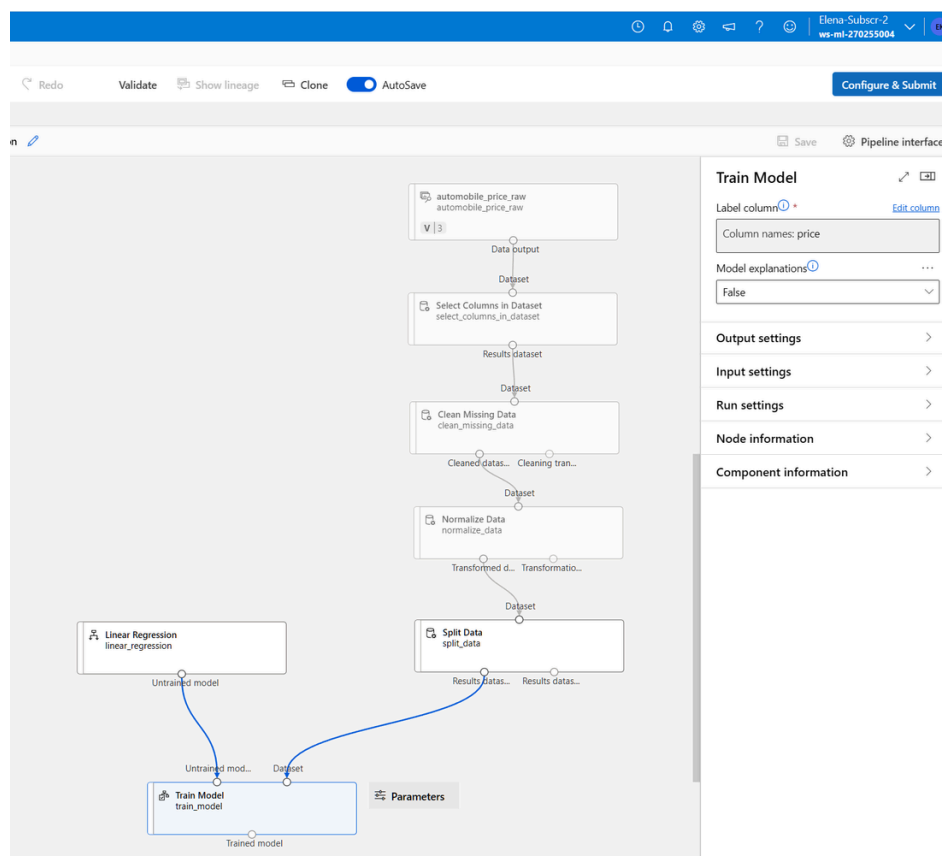
Component information >

Model Training & Evaluation

Model Selection and Training

Step	What to Do	How to Do It	Why It's Important
1.	Select Linear Regression	Drag and drop the "Linear Regression" module onto the canvas. This represents the chosen algorithm.	
2.	Train Model	- Connect the "Linear Regression" module's output to the "Train Model" module's "Untrained model" input. - Connect the left output ("Results dataset" for training data) of the "Split Data" module to the "Train Model" module's "Dataset" input.	Label column: price.

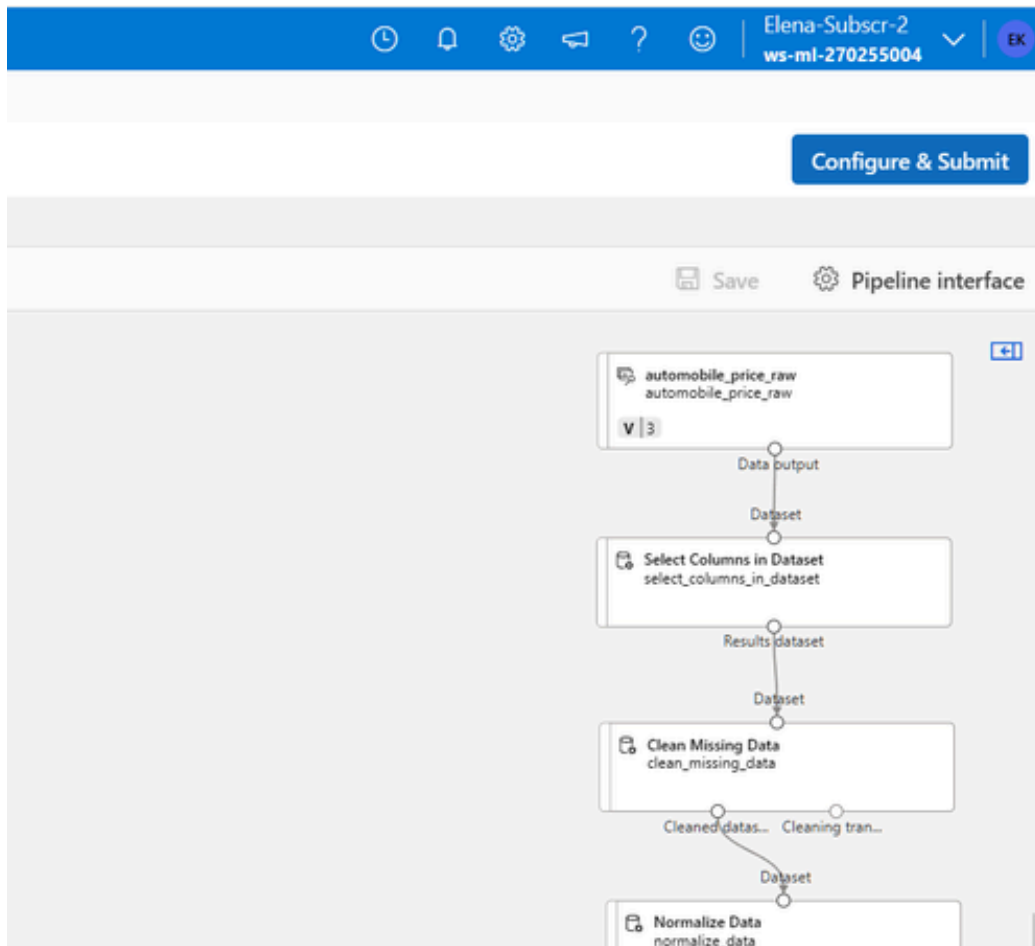
Screenshot 66: Train Model settings

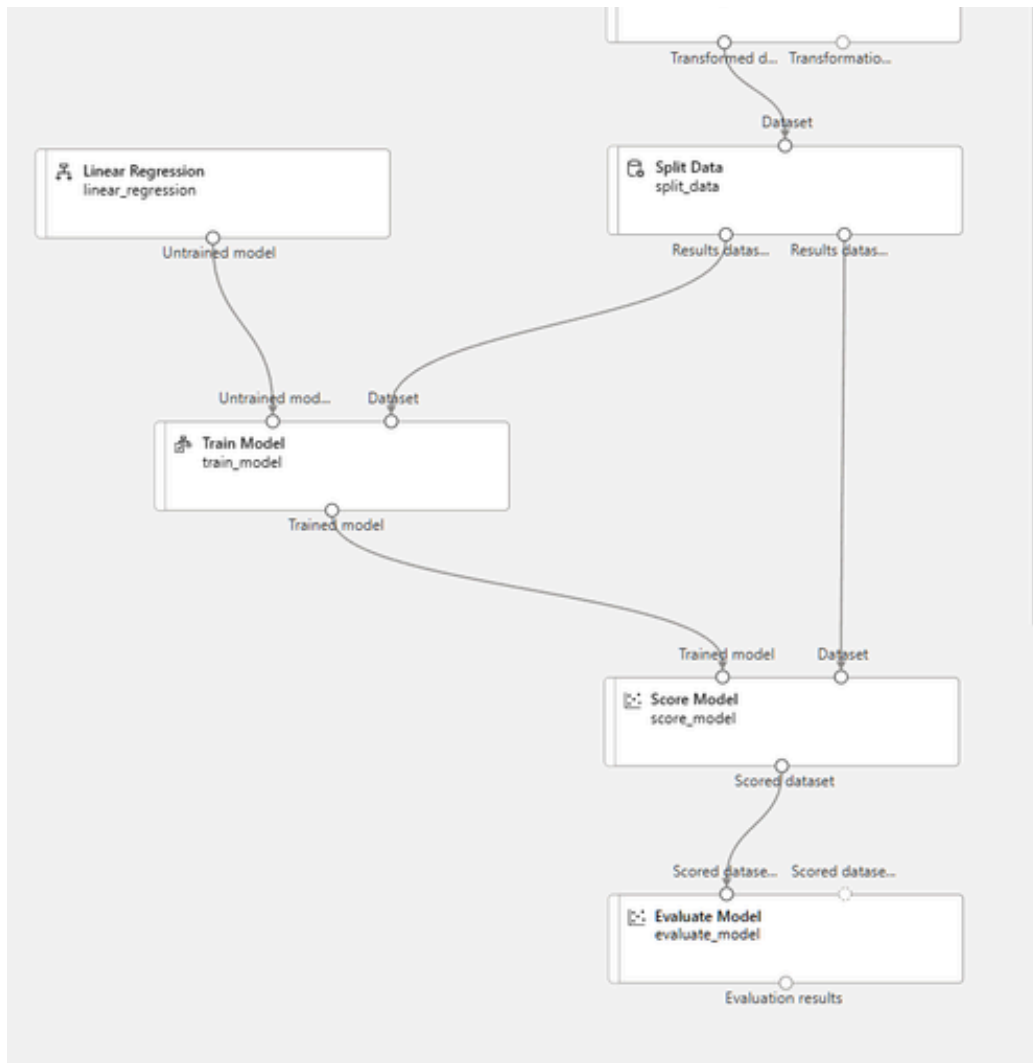


Scoring and Evaluation

3.	Score Model	- Connect the "Trained model" output from the "Train Model" module to the "Score Model" module's "Trained model" input (left). - Also, connect the right output ("Results dataset" for test data) from "Split Data" to the "Score Model" module's "Dataset" input (right).	
4.	Evaluate Model	Connect the output from the "Score Model" module to an "Evaluate Model" module. This module will calculate performance metrics.	This step is crucial for understanding the model's accuracy. It provides key regression metrics (like MAE, RMSE, R^2) to quantitatively assess how well your model predicts prices and determine if further tuning is needed.

Screenshot 67: The finished pipeline

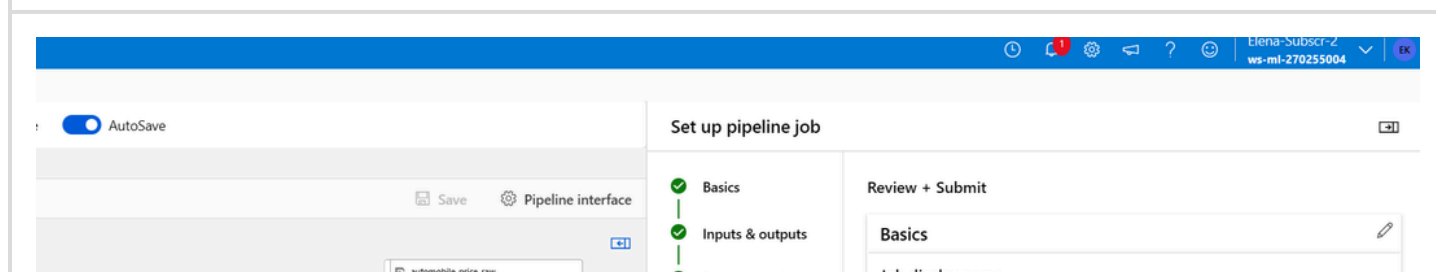




Validation and Submission

Step	What to Do	How to Do It	Settings
1.	Validate and Submit Pipeline	Click " Validate " (if available) and then "Configure & Submit".	
		Create Compute	- New experiment name : price-train-270255004 - Default compute Add a "Create Azure ML Cluster" module. Configure settings like: - Dedicated priority -Max 2 nodes -120s idle scale down - -Standard_DS11_v2 CPU size -Select compute type: Compute cluster Create compute cluster: Location*Australia East Dedicated CPU Standard DS3-v2 Compute name testing-cluster-270255004 Add tags Location : nz Select Azure ML Compute Cluster: testing-cluster-270255004, Succeeded, Standard_DS3_v2, 4 vCPUs (cores), 14 GB, 28 GB (storage), \$--/hr, Nodes: 0 idle, 0 busy, 1 unprovisioned Review + Submit

Screenshot 68: Submit settings



Runtime settings

Job description
Pipeline created on 20251120

Experiment
price-train-270255004

Tags
No tags

Inputs & outputs

Inputs
None

Outputs
None

Runtime settings

Continue on step failure
True

Default compute
testing-cluster-270255004

Default datastore
workspaceblobstore

Submit Back Next Close

Check
Submission
Process

Select **Pipelines** in the
left navigation pane.

Find the new pipeline job (e.g.,
car_price_prediction).

Screenshot 69: Submitted pipeline job

Azure AI | Machine Learning Studio

Default Directory > ws-ml-270255004 > Pipelines

Pipelines

Pipeline jobs Pipeline endpoints Pipeline drafts

+ New pipeline Refresh Reset view

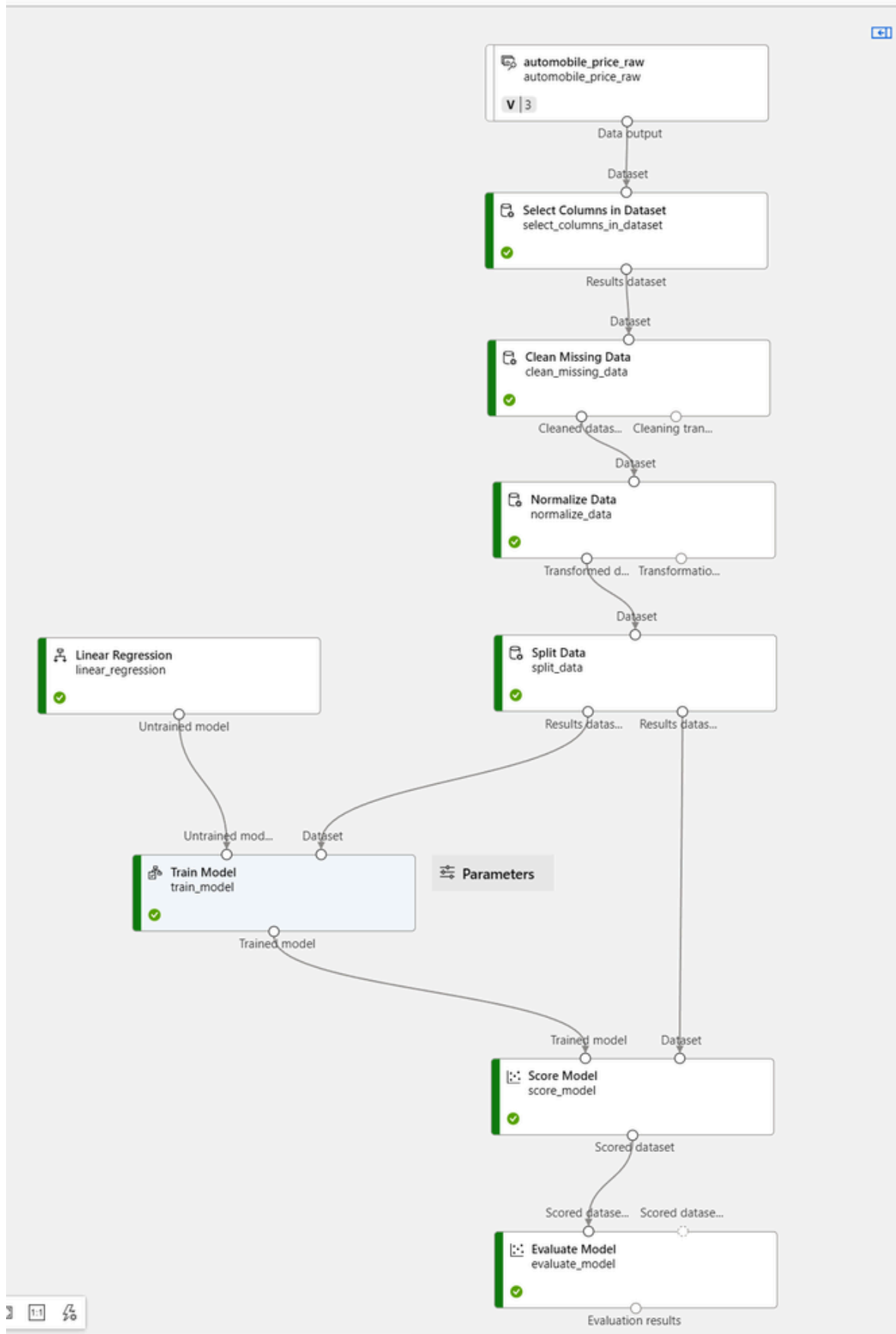
Search Filter Columns

Display name	Experiment	Status	Description	Created on	Duration	Created by	Tags
car_price_prediction	price-train-270255004	Running	Pipeline created on 20...	Nov 20, 2025 2:52 PM	24s	Elena Kroupkin	

Screenshot 70: Validated and submitted

View profiling Publish Schedule Show lineage Show compare list Cancel

Share Add to compare Job overview

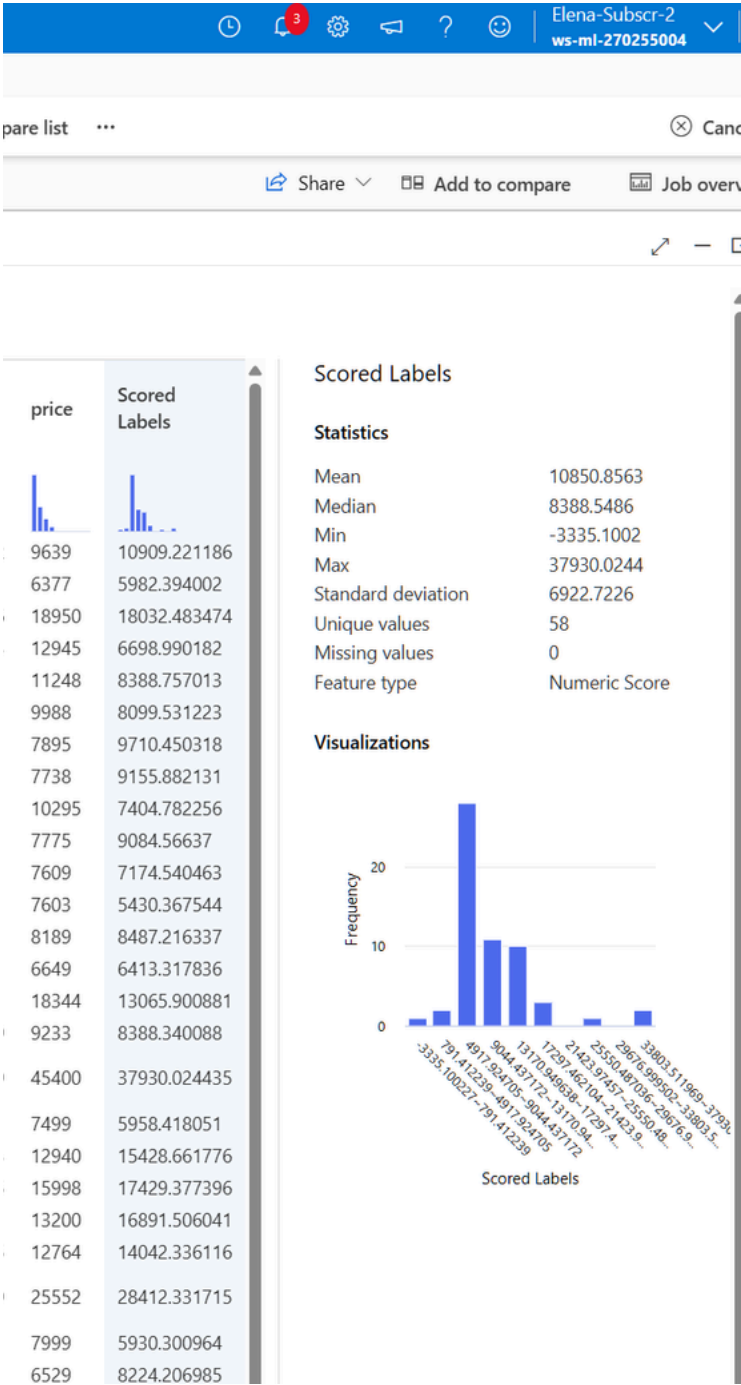


Check Model's outcome

Review Scored Data and Evaluation Results

Step	What to Do	How to Do It	Why It's Important
1.	Check Scored Data	Right-click the "Score Model" module → "Preview data" → "Scored dataset".	Compare the "Scored Labels" column with the original "price" column to observe predictions. <i>(Note: Predicted prices may differ from actuals, potentially due to limited data or model limitations.)</i>

Screenshot 71: Preview Scored dataset



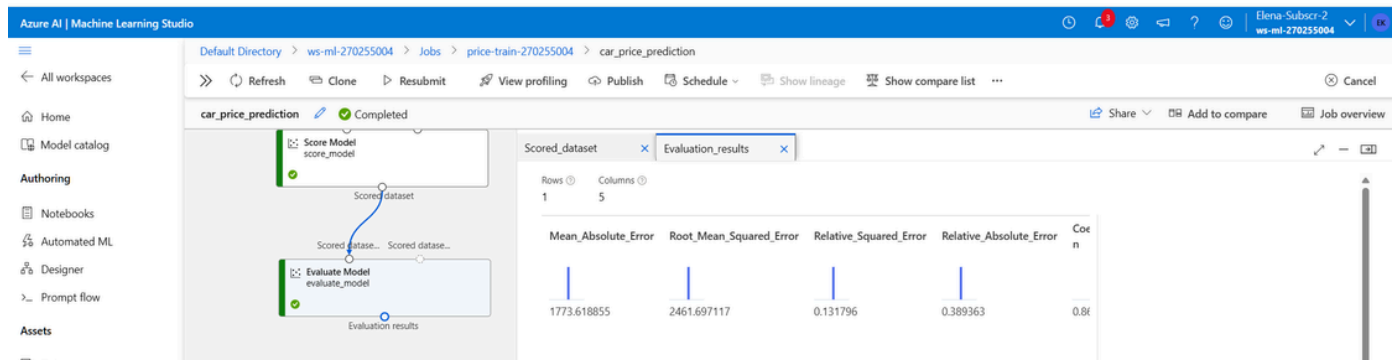
2.	Check	Right-click the "Evaluate	Review evaluation metrics such as :
----	-------	---------------------------	-------------------------------------

**Evaluated
data**

Model" module →
"Preview data" →
"Evaluation results"

- Mean_Absolute_Error
- Root_Mean_Squared_Error
- Relative_Squared_Error
- Relative_Absolute_Error
- Coefficient_of_Determination

Screenshot 72: Evaluated data

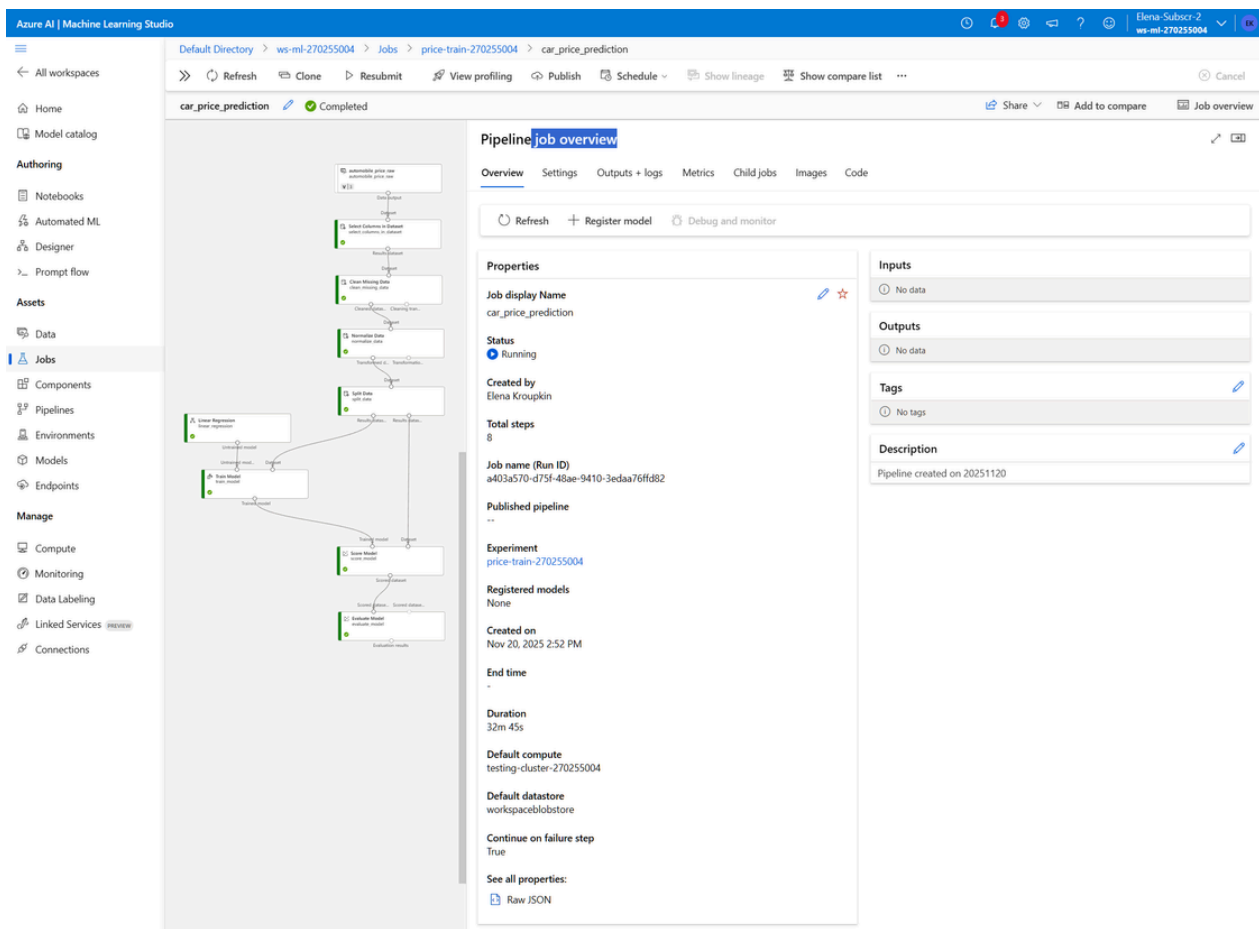


3.

**Overall Job
information**

Click **job overview**
button on right top
corner

Screenshot 73: Pipeline job overview



Explanation for Each Regression Metric

Detailed Metric Explanations

Here are the refined explanations for each regression metric. I've adjusted the "Statistics" presentation to be more typical for metric reporting (just the value, as the other stats like min/max/std dev for a single metric value are less relevant).

Mean Absolute Error (MAE)

MAE measures the average magnitude of errors in a set of predictions, without considering their direction. It indicates how close the predictions are to the actual values on average.

Explanation: The model's average prediction for a car price is off by approximately \$1773.62. Lower MAE is better.

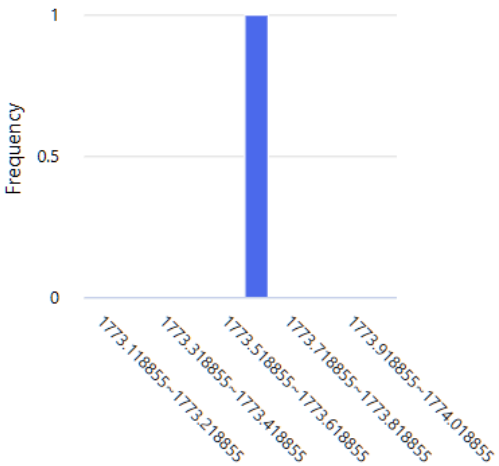
Screenshot 74: Mean_Absolute_Error graph

Mean_Absolute_Error

Statistics

Mean	1773.6189
Median	1773.6189
Min	1773.6189
Max	1773.6189
Standard deviation	-
Unique values	1
Missing values	0
Feature type	Numeric Feature

Visualizations

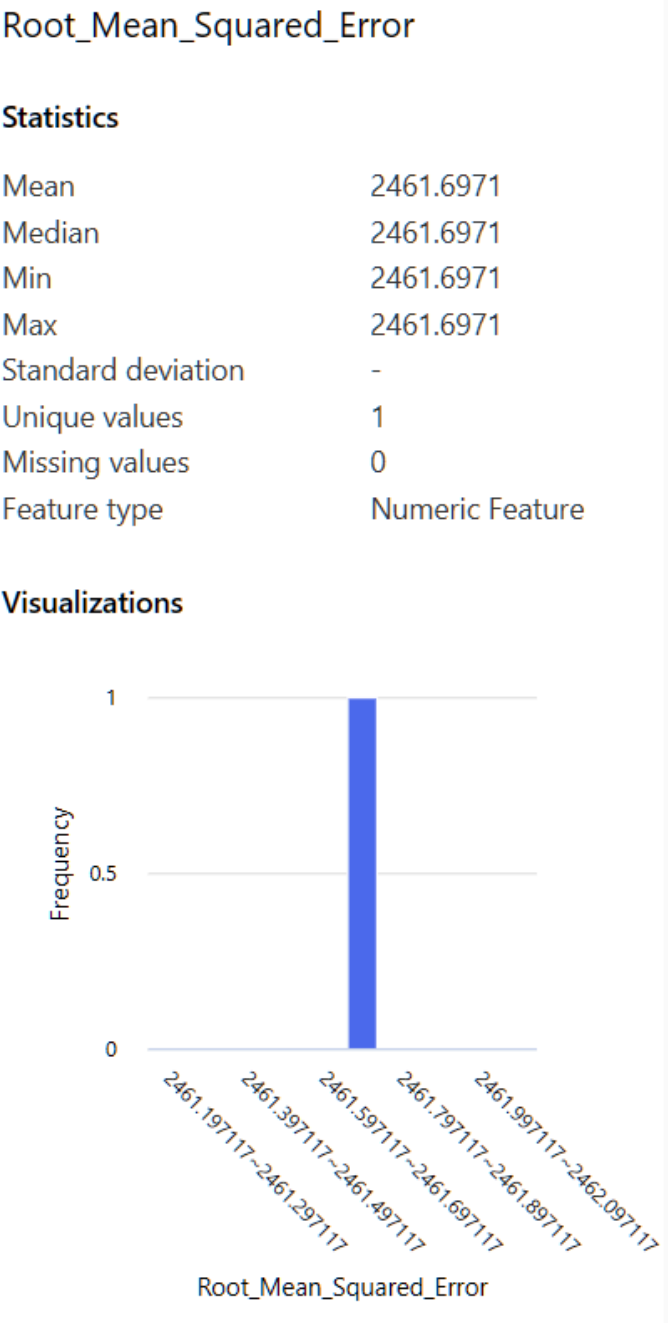


Root Mean Squared Error (RMSE)

RMSE is the square root of the average of the squared errors. It penalizes larger errors more heavily than MAE, making it useful when significant errors are particularly undesirable.

Explanation: The model's typical error, giving more weight to larger mistakes, is about \$2461.70. Lower RMSE is better.

Screenshot 75: Root_Mean_Squared_Error graph



Relative Squared Error

Relative Squared Error normalizes the total squared error by the total squared error of a naive model (predicting the mean). It provides a scale-independent measure of error, often reflecting how much better the model is than a baseline.

Explanation: A value of 0.1318 means the model's squared errors are about 13.18% of what they would be if you just predicted the average car price. Lower is better, with 0 being a perfect model.

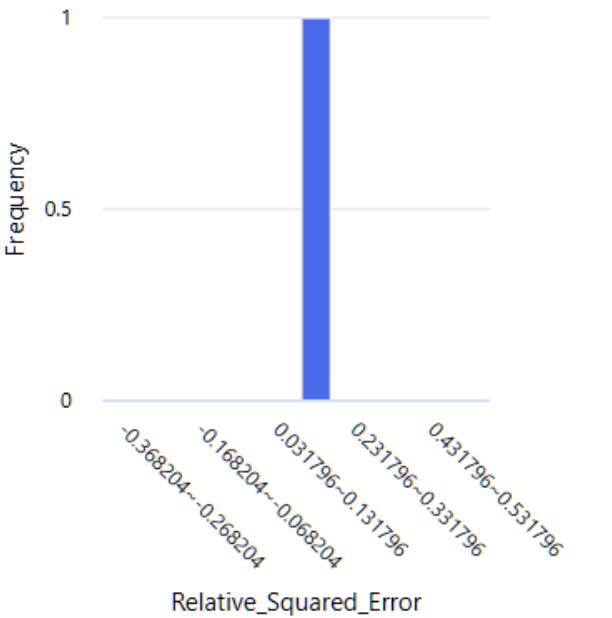
Screenshot 76: Relative_Squared_Error graph

Relative_Squared_Error

Statistics

Mean	0.1318
Median	0.1318
Min	0.1318
Max	0.1318
Standard deviation	-
Unique values	1
Missing values	0
Feature type	Numeric Feature

Visualizations



Relative Absolute Error

Relative Absolute Error normalizes the total absolute error by the total absolute error of a naive model (predicting the mean). Like RSE, it offers a scale-independent way to compare models.

Explanation: A value of 0.3894 means the model's absolute errors are about 38.94% of what they would be if you just predicted the average car price. Lower is better.

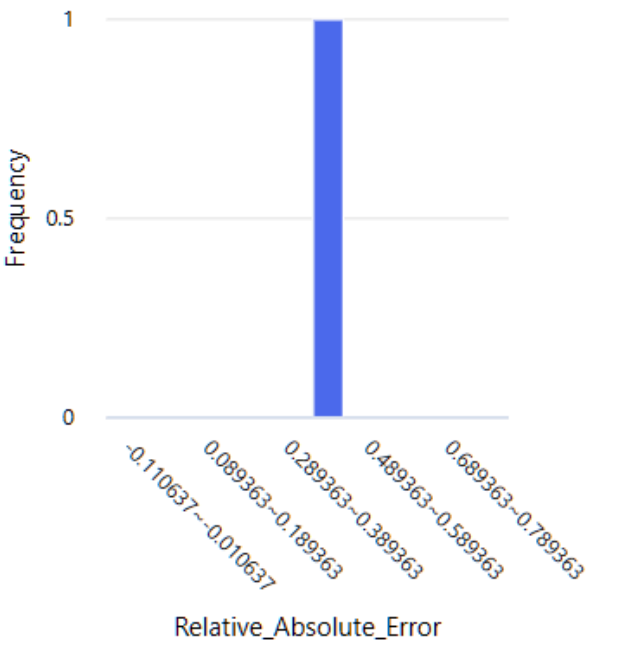
Screenshot 77: Relative_Absolute_Error graph

Relative_Absolute_Error

Statistics

Mean	0.3894
Median	0.3894
Min	0.3894
Max	0.3894
Standard deviation	-
Unique values	1
Missing values	0
Feature type	Numeric Feature

Visualizations



Coefficient of Determination (R-squared)

R-squared represents the proportion of the variance in the dependent variable (car price) that the model can predict from the independent variables. It indicates how well the model fits the observed data.

Explanation: The model explains approximately 86.82% of the variance in car prices. Values closer to 1 (or 100%) indicate a better fit, meaning the features are very effective at predicting the price.

Screenshot 78: Coefficient_of_Determination graph

Coefficient_of_Determination

Statistics

Mean	0.8682
Median	0.8682
Min	0.8682
Max	0.8682
Standard deviation	-
Unique values	1
Missing values	0
Feature type	Numeric Feature

Visualizations

